CETI Plantel Colomos

Aseguramiento de la Calidad de Software

CoProof
Documentos Primer Parcial

Profesor:
Luis Fernando Gaspar Pérez

Autores:
David Alejandro López Torres - 22310432
Daniel Tejeda Saavedra 22310431
Emiliano Flores Márquez 22110044

Guadalajara, México

19 de marzo de 2026

Índice

1. Historias de usuario del sistema CoProof **3**
1.1. Historias de autenticación y acceso . . . 3
1.2. Historias de gestión de proyectos . . . 4
1.3. Historias de sesión abierta . . . 5
1.4. Historias de validación . . . 5

2. Arquitectura **6**
2.1. Visión general . . . 6
2.2. Acrónimos y convención de notación . . . 6
2.3. Visión de flujo extremo a extremo . . . 7
2.4. Componentes y responsabilidades . . . 8
2.4.1. Backend Flask . . . 8
2.4.2. LeanServer . . . 8
2.4.3. Integración con GitHub . . . 8
2.5. Patrones de diseño aplicados . . . 8
2.5.1. Arquitectura por capas . . . 8
2.5.2. Application Factory . . . 8
2.5.3. Productor–consumidor con colas . . . 9
2.5.4. Adapter/Gateway de integraciones . . . 9
2.5.5. Transaction Script sobre Git . . . 9
2.5.6. Control de concurrencia . . . 9
2.5.7. Propagación de estado en árbol de nodos . . . 9
2.5.8. Sincronización orientada a eventos . . . 9
2.6. Flujos operativos principales . . . 9
2.6.1. Autenticación . . . 9
2.6.2. Creación de proyecto . . . 9
2.6.3. Validación de nodo (solve) . . . 10
2.7. Modelo de dominio . . . 10
2.7.1. Entidades principales . . . 10
2.7.2. Relaciones clave . . . 10

3. Diagrama de clases **11**
3.1. Diagrama de clases del servidor (Backend Core) . . . 11
3.2. Capa de runtime y configuración . . . 11
3.3. Capa de API . . . 11
3.4. Capa de servicios . . . 11
3.5. Modelos y esquemas . . . 12
3.6. Diagrama de clases del Lean Worker (microservicio) . . . 14
3.7. Inicialización y enrutamiento . . . 14
3.8. Servicios . . . 14
3.9. Gestión de entorno temporal . . . 14
3.10. Normalización . . . 15
3.11. Diagrama de clases del Frontend SPA . . . 17

3.12. Arquitectura de interfaz y composición de rutas 17
3.13. Componentes de dominio y operaciones principales 17
3.14. Estado asíncrono y observabilidad en tiempo real 17
3.15. Capa de comunicaciones y seguridad 18

4. Diagrama de calles **20**
4.1. Cliente a Backend Core 20
4.2. Backend Core a Lean Worker 20
4.3. Resolución y retroalimentación 20
4.4. Escalabilidad 21

Índice de tablas

Índice de figuras

3.1. Diagrama de clases del servidor 13
3.2. Diagrama de clases del Lean Server (microservicio) 16
3.3. Diagrama de clases del Frontend 19
4.1. Diagrama de calles V1 (general) 22
4.2. Diagrama de calles V2 (de flujos) 23

1. Historias de usuario del sistema CoProof

Las historias de usuario presentadas a continuación articulan los requisitos funcionales de CoProof desde la perspectiva del usuario final. Cada historia sigue un formato estructurado que incluye el actor ("Como"), la capacidad deseada ("quiero") y el beneficio esperado ("para"), junto con criterios de aceptación explícitos que representan el contrato de calidad que el sistema debe cumplir. Esta estructura permite al equipo de desarrollo priorizar e iterar sobre capacidades de forma coherente, asegurando que cada incremento incremente valor percibido y mantenga consistencia de experiencia de usuario.

1.1. Historias de autenticación y acceso

Título	Registro de cuenta
Historia de usuario	**Como** persona nueva en la plataforma, **quiero** crear una cuenta con correo verificable, **para** acceder al sistema con identidad propia.
Criterios de aceptación	☐ El usuario completa formulario con teclado físico, teclado en pantalla o comando de voz asistido, y confirma con clic, toque o tecla Enter. ☐ Si falta un campo obligatorio, la interfaz muestra error en línea bajo el campo y un banner superior con resumen de errores. ☐ Si el correo ya existe, el sistema abre un diálogo modal con la opción de ir a iniciar sesión. ☐ La verificación se confirma por correo electrónico y se refleja en un panel de estado integrado en la vista de registro. ☐ Al finalizar, aparece un toast de éxito y el sistema redirige a la pantalla de inicio de sesión.

Título	Inicio de sesión
Historia de usuario	**Como** usuario registrado, **quiero** iniciar sesión con correo y contraseña, **para** usar funciones que requieren autenticación.
Criterios de aceptación	☐ El usuario ingresa credenciales con teclado físico, teclado en pantalla o comando de voz y envía con botón, Enter o comando "iniciar". ☐ Si las credenciales son correctas, el sistema abre el panel principal y muestra un toast de bienvenida con el rol activo. ☐ Si las credenciales son incorrectas, aparece un mensaje en línea bajo contraseña y un contador de intentos en un panel informativo. ☐ Tras 3 intentos fallidos, el sistema muestra diálogo de bloqueo temporal y acceso directo a recuperación de cuenta. ☐ Si hay error de conexión, el sistema muestra un banner persistente con opción "Reintentar".

1.2. Historias de gestión de proyectos

Título	Creación de proyecto privado
Historia de usuario	**Como** usuario autenticado, **quiero** crear un proyecto privado, **para** trabajar con acceso restringido a colaboradores seleccionados.
Criterios de aceptación	☐ El usuario abre "Nuevo proyecto", selecciona "Privado" y completa campos con teclado, ratón o pantalla táctil. ☐ El sistema valida campos requeridos y muestra errores en línea antes de permitir confirmar. ☐ Al crear, el sistema asigna al creador como líder y muestra diálogo de confirmación con identificador del proyecto. ☐ La invitación de colaboradores se realiza desde un panel lateral con búsqueda por correo o nombre.

1.3. Historias de sesión abierta

Título	Apertura de proyecto en sesión individual
Historia de usuario	**Como** usuario con acceso autorizado, **quiero** abrir un proyecto en sesión individual, **para** trabajar en modo personal sobre el grafo.
Criterios de aceptación	☐ El usuario selecciona un proyecto con doble clic, toque o comando "abrir en sesión individual". ☐ El sistema valida permisos y carga la vista del proyecto con indicador de modo en la barra superior. ☐ Si el acceso es denegado, se muestra diálogo modal con motivo y opción para solicitar acceso.

1.4. Historias de validación

Título	Solicitud de demostración de nodo a un agente
Historia de usuario	**Como** usuario, **quiero** solicitar a un agente una demostración completa de un nodo, **para** evaluar una solución automática.
Criterios de aceptación	☐ El usuario inicia la solicitud desde botón "Pedir demostración" en el panel del nodo. ☐ El sistema muestra cola y progreso en la consola integrada con estados: en cola, ejecutando, validando, finalizada. ☐ Si el agente entrega una propuesta, el sistema la presenta en el visor y muestra resultado de validación formal.

Título	Guardado de cambios en sesión individual
Historia de usuario	**Como** usuario en sesión individual, **quiero** guardar cambios del proyecto, **para** crear una solicitud de autorización para el líder.
Criterios de aceptación	☐ El usuario activa "Guardar cambios" con clic, toque, `Ctrl+S` o comando de voz habilitado. ☐ El sistema compila un resumen de cambios y lo muestra en un diálogo de confirmación antes del envío. ☐ Al confirmar, el sistema registra la solicitud y muestra identificador en un toast y en panel de actividad.

2. Arquitectura

2.1. Visión general

La plataforma CoProof está diseñada bajo un enfoque de **Arquitectura Orientada a Servicios (SOA, *Service-Oriented Architecture*)** con una transición progresiva hacia **microservicios** especializados. La decisión técnica central consiste en desacoplar las operaciones administrativas de las operaciones de validación formal asíncrona, debido a que el dominio del problema —la verificación formal de teoremas y la compilación de proyectos matemáticos en Lean4— presenta cargas de trabajo costosas, de duración variable y con picos de latencia difíciles de predecir. En este contexto, un diseño monolítico síncrono degradaría rápidamente la experiencia de usuario cuando múltiples validaciones se ejecutan de forma concurrente.

El principio operativo principal es el **procesamiento asíncrono impulsado por eventos**. En términos prácticos, el **Backend Core** administra estado y reglas de negocio, mientras que el **Lean Worker** procesa tareas de compilación y validación formal fuera del ciclo de petición-respuesta inmediato. Esta separación se alinea con una aplicación de **segregación de responsabilidades de consulta y comando (CQRS, *Command Query Responsibility Segregation*) a nivel arquitectónico**: las rutas transaccionales y de lectura del dominio permanecen disponibles incluso cuando existen validaciones largas en cola.

La plataforma adopta una comunicación basada en mensajes mediante el patrón **publicación/suscripción (Pub/Sub, *Publish/Subscribe*)** y colas de trabajo. En lugar de acoplar Backend Core y Lean Worker con llamadas directas **RPC (*Remote Procedure Call*)** o flujos **HTTP (*HyperText Transfer Protocol*)** síncronos, se utiliza Redis como *broker* de mensajes y almacén de resultados. Esta estrategia proporciona tolerancia a fallos: si un trabajador cae, la tarea no se pierde, sino que permanece encolada hasta que un proceso de worker vuelve a estar en línea y la consume. Además, el uso de identificadores de trabajo permite observabilidad y trazabilidad extremo a extremo.

Otro componente estructural es el **Gateway de Integración Externa**. CoProof no opera como almacenamiento aislado del código fuente, sino como orquestador entre: (1) un almacenamiento relacional para metadatos y estado del grafo de demostración, y (2) un motor externo de control de versiones (GitHub) que conserva el historial inmutable de cambios colaborativos. En consecuencia, la plataforma separa responsabilidades entre persistencia del estado lógico interno y versionado distribuido del contenido formal, evitando inconsistencias entre colaboración matemática y gestión de repositorios.

2.2. Acrónimos y convención de notación

Para mantener consistencia en todo el documento, se utiliza la siguiente notación estable:

- **API**: Interfaz de Programación de Aplicaciones (*Application Programming Interface*).
- **SOA**: Arquitectura Orientada a Servicios.

- **CQRS**: Segregación de responsabilidades de consulta y comando.
- **SPA**: Aplicación de Página Única (*Single-Page Application*).
- **SSE**: Eventos Enviados por el Servidor (*Server-Sent Events*).
- **REST**: Transferencia de Estado Representacional (*Representational State Transfer*).
- **JSON**: Notación de Objetos de JavaScript (*JavaScript Object Notation*).
- **ORM**: Mapeo Objeto-Relacional (*Object-Relational Mapping*).
- **CI/CD**: Integración Continua y Entrega/Despliegue Continuo.
- **JWT**: JSON Web Token, esquema de token firmado para autenticación.
- **OAuth**: protocolo abierto de autorización delegada.
- **UUID**: Identificador Universalmente Único (*Universally Unique Identifier*).
- **NL2FL**: traducción de lenguaje natural a lenguaje formal (*Natural Language to Formal Language*).

Cuando se mencione **Backend Core**, se refiere al servidor principal de orquestación del dominio. Cuando se mencione **Lean Worker**, se refiere al microservicio validador formal. Cuando se mencione **Frontend SPA**, se refiere al cliente web de la plataforma. Esta convención se conserva de forma uniforme en todas las secciones y en la interpretación de los diagramas. En el alcance actual de los diagramas no se modela un microservicio explícito de cómputo exhaustivo.

2.3. Visión de flujo extremo a extremo

El flujo arquitectónico completo puede resumirse así: una acción de usuario inicia en el Frontend SPA, llega al Backend Core por REST/JSON, se valida contra reglas del dominio y, si requiere verificación formal, se transforma en un mensaje de tarea enviado al broker Redis. El Lean Worker consume esa tarea, ejecuta Lean4 en un entorno controlado, transforma la salida en JSON estandarizado y la deposita en el backend de resultados. Posteriormente, el Backend Core expone el avance y estado final al Frontend SPA, incluyendo notificaciones en tiempo real por SSE. Este ciclo evita bloqueos en el servidor principal, mejora la escalabilidad horizontal y preserva una experiencia interactiva incluso ante compilaciones costosas.

En términos de evolución, la topología permite incorporar nuevos microservicios (por ejemplo, módulos de NL2FL o agentes de IA) sin romper la orquestación existente: basta con definir nuevas colas y agregar adaptadores cliente en el Backend Core. Por ello, la arquitectura no está cerrada a un único tipo de validación matemática, sino preparada para un ecosistema de servicios especializados que comparten contratos de mensajería consistentes.

2.4. Componentes y responsabilidades

2.4.1. Backend Flask

Actúa como punto de entrada HTTP y coordinador entre base de datos, GitHub y LeanServer.

- expone endpoints de auth, proyectos, nodos y webhooks,
- aplica reglas de negocio en servicios (`ProjectService`, `AuthService`, `CompilerClient`),
- persiste estado de dominio en PostgreSQL (`NewProject`, `NewNode`, `User`),
- usa Redis/Celery para ejecución asíncrona.

2.4.2. LeanServer

Se encarga de la validación formal de código Lean en aislamiento.

- consume tareas `tasks.verify_snippet` y `tasks.verify_project_files`,
- ejecuta Lean sobre archivos temporales,
- parsea mensajes de compilación,
- retorna una respuesta normalizada (`valid`, `errors`, tiempos y conteos).

2.4.3. Integración con GitHub

Se utiliza para autenticación OAuth y como repositorio remoto del contenido Lean.

- OAuth para obtener identidad/token del usuario,
- API REST para crear repositorios, actualizar archivos y gestionar PRs,
- webhook `push` para disparar reindexado asíncrono.

2.5. Patrones de diseño aplicados

2.5.1. Arquitectura por capas

Organización en capas con dependencias dirigidas, donde cada capa expone contratos y concentra una responsabilidad: API (entrada/salida HTTP), servicios (orquestación), dominio/datos (estado persistente) e infraestructura (tecnología externa). Regla operativa: API → Servicios → Dominio/Infraestructura.

2.5.2. Application Factory

`create_app` centraliza creación de la aplicación y registro de extensiones/blueprints, concentrando la configuración por entorno.

2.5.3. Productor–consumidor con colas

El backend publica tareas y workers especializados las consumen. Productores (`CompilerClient`), Consumidor Lean (`lean_queue`), Broker (**Redis**).

2.5.4. Adapter/Gateway de integraciones

`CompilerClient` abstrae RPC por Celery, `AuthService` encapsula llamadas a GitHub.

2.5.5. Transaction Script sobre Git

`read_only_worktree(...)` para validación sin persistencia, `git_transaction(...)` para edición y commit/push.

2.5.6. Control de concurrencia

`acquire_project_lock` y `acquire_branch_lock` evitan condiciones de carrera en operaciones Git concurrentes.

2.5.7. Propagación de estado en árbol de nodos

El estado de `NewNode` (`sorry`, `validated`) se propaga por reglas: una solución válida marca el nodo como `validated`, un split devuelve nodos a `sorry` hasta completar subpruebas.

2.5.8. Sincronización orientada a eventos

El webhook de GitHub (`push`) dispara reindexado asíncrono (`async_reindex_project`) con validación HMAC.

2.6. Flujos operativos principales

2.6.1. Autenticación

1. Frontend solicita URL OAuth (`GET /api/v1/auth/github/url`).
2. Usuario autoriza y frontend recibe `code`.
3. Frontend envía `code` a backend (`POST /api/v1/auth/github/callback`).
4. `AuthService` intercambia token y emite JWT interno.
5. Renovación con refresh token (`POST /api/v1/auth/refresh`).

2.6.2. Creación de proyecto

1. Frontend envía `POST /api/v1/projects`.
2. `ProjectService` valida datos y token GitHub.
3. Se valida contexto Lean del objetivo con `CompilerClient.verify_snippet`.

4. Se crea repositorio remoto con archivos iniciales.
5. Se persisten `NewProject` y nodo raíz `NewNode`.

2.6.3. Validación de nodo (solve)

1. Frontend envía POST/api/v1/projects/{project_id}/nodes/{node_id}/solve.
2. `nodes.solve_node` reconstruye contexto por árbol de imports.
3. Se verifica con LeanServer.
4. Si es válido, se crea rama y pull request.

2.7. Modelo de dominio

2.7.1. Entidades principales

- `NewProject`: datos del proyecto, configuración de objetivo, repositorio remoto y colaboradores.
- `NewNode`: unidad de trabajo en el DAG lógico, con jerarquía padre-hijo y estado.
- `User`: identidad local y vínculo OAuth/token con GitHub.

2.7.2. Relaciones clave

- `NewProject` 1..* `NewNode`.
- `NewNode` tiene autorrelación (`parent` / `children`).
- `User` se vincula a proyectos por autoría y colaboración.

3. Diagrama de clases

3.1. Diagrama de clases del servidor (Backend Core)

El diagrama de clases **class-01** representa al Backend Core como cerebro administrativo de la plataforma. Su propósito principal no es ejecutar lógica matemática de Lean4, sino coordinar recursos de dominio, validar permisos, mantener consistencia transaccional y actuar como punto de integración entre clientes, base de datos y sistema de colas. Esta distinción funcional es crítica: al evitar ejecutar validaciones formales prolongadas dentro del proceso principal, el servidor conserva capacidad de respuesta para autenticación, navegación de grafos y operaciones de colaboración.

3.2. Capa de runtime y configuración

El diseño del runtime utiliza un patrón **Factory** mediante `AppFactory`, apoyado por un `ConfigResolver` para inyectar configuración por entorno. El diagrama distingue explícitamente una configuración de pruebas (`TestingConfig`, con SQLite en memoria) y una configuración productiva (`ProductionConfig`, con PostgreSQL), lo que respalda un pipeline de CI/CD robusto y reproducible. Esta separación de entornos reduce riesgos de regresión al permitir pruebas rápidas y aisladas sin depender de infraestructura externa persistente.

Adicionalmente, un `ExtensionRegistry` centraliza extensiones transversales del servicio: **SQLAlchemy** como ORM, **Alembic** para migraciones de esquema y extensiones de autenticación basadas en JWT. Este registro evita inicializaciones dispersas y mejora la mantenibilidad porque concentra el punto de acoplamiento técnico en una sola capa de arranque.

3.3. Capa de API

La capa de API sigue un enrutamiento estandarizado y expone recursos orientados al dominio de pruebas formales. En particular, `NodesController` concentra endpoints de alto valor funcional, como `solve_node`, `split_node` y `verify_node_import_tree`. La presencia de estas operaciones en el diagrama evidencia que el servidor no gestiona únicamente entidades CRUD tradicionales, sino transformaciones sobre estructuras jerárquicas de metas, submetas y dependencias de importación.

Desde la perspectiva arquitectónica, estas rutas constituyen una interfaz semántica sobre grafos de demostración. Es decir, cada endpoint no solo muta registros, sino que preserva invariantes del árbol de prueba colaborativo: coherencia entre nodos padre e hijo, control de prerrequisitos y estados de validación compatibles con la estrategia de compilación formal.

3.4. Capa de servicios

La lógica de negocio vive en servicios de dominio. `LeanService` prepara cargas útiles de verificación mediante `build_verify_payload` y normaliza importaciones por `normalize_lean_imports`

antes de delegar la validación formal asíncrona. Esta prevalidación protege al sistema de inconsistencias sintácticas y evita enviar al worker datos mal estructurados.

Para asincronía, el diagrama muestra el patrón **Client/Adapter** con `CompilerClient`. Este cliente encapsula la complejidad de encolado en Redis vía `dispatch_task` y devuelve un identificador de trabajo que sirve para rastreo posterior. El beneficio es doble: desacopla el backend de la implementación concreta del worker y permite evolucionar la infraestructura de ejecución sin rediseñar los controladores REST.

3.5. Modelos y esquemas

En la capa de persistencia, destaca `GraphNodeModel` con identificadores UUID, un campo de clasificación de nodo (`node_type`: `global`, `goal`, `theorem`, `lemma`) y relaciones jerárquicas complejas (`parent`, `children`, `prerequisites`). Esta modelación facilita construir árboles de demostración colaborativos donde cada nodo representa una pieza verificable y trazable del proceso formal.

Los *schemas* (implementados con Marshmallow) actúan como capa anticorrupción para serialización y deserialización hacia el Frontend SPA. En otras palabras, aíslan el modelo interno del contrato externo, permitiendo cambios internos en persistencia sin romper la interfaz de consumo.

En conjunto, **class-01** muestra una arquitectura backend madura: configuración desacoplada por entorno, controladores orientados al dominio, servicios de normalización y adaptación asíncrona, y una capa de datos capaz de representar colaboraciones matemáticas complejas de forma consistente.

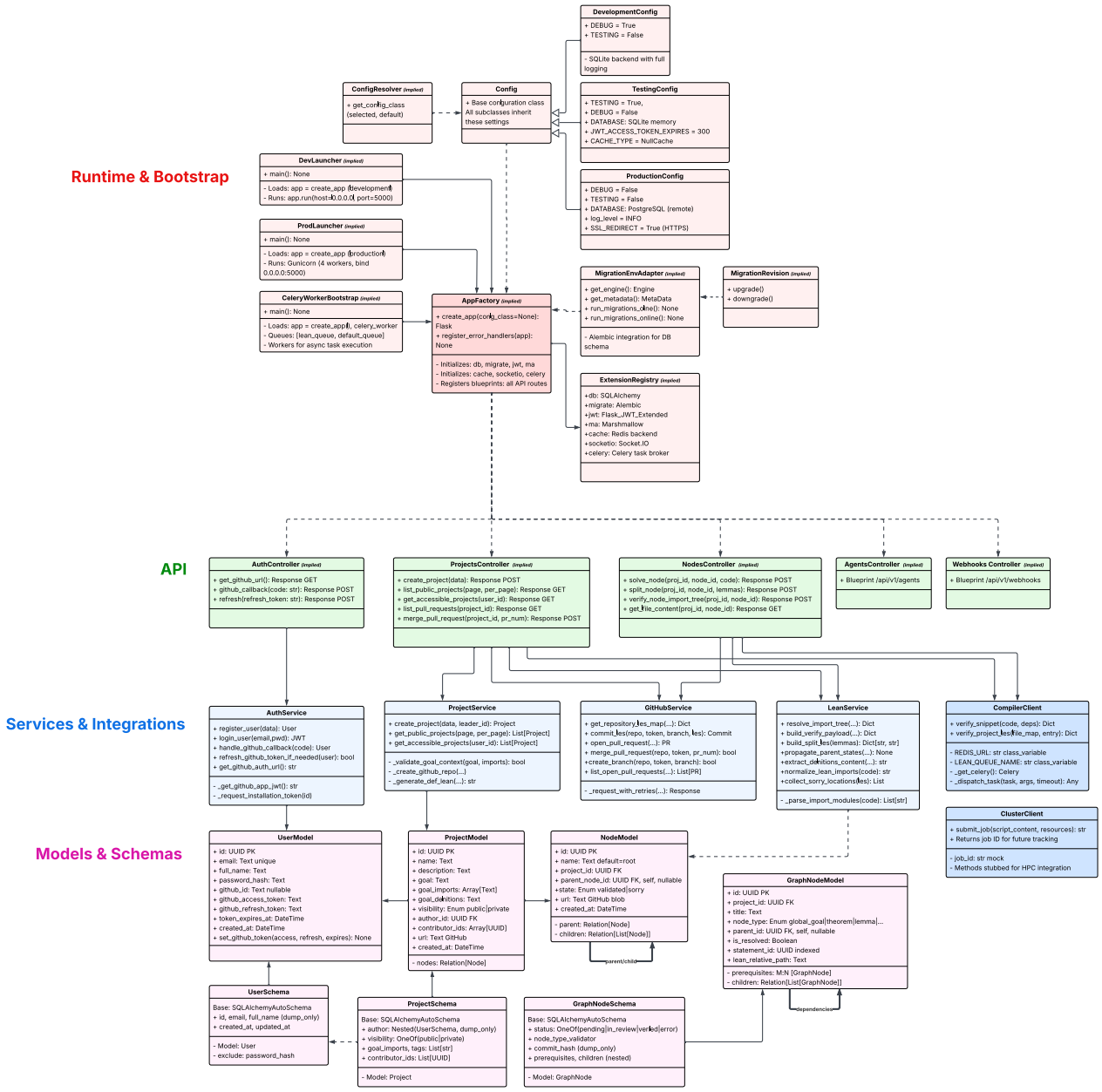

Figura 3.1: Diagrama de clases del servidor

3.6. Diagrama de clases del Lean Worker (microservicio)

El diagrama **class-02** describe el microservicio Lean Worker como un proceso **sin estado** orientado a validación formal. A diferencia del Backend Core, este componente no expone puertos HTTP ni administra sesiones de usuario; su responsabilidad es consumir tareas de verificación, ejecutar Lean4 de forma controlada y devolver resultados estructurados. Esta separación reduce superficie de fallo y simplifica el escalado horizontal: agregar capacidad equivale a instanciar más workers escuchando la misma cola.

Es importante precisar el alcance: en la versión actual de los diagramas, el Lean Worker representa el **validador formal** y no un microservicio independiente de cómputo exhaustivo. Si en una iteración futura se añade ese tipo de servicio, deberá modelarse como componente adicional con su propia cola, contratos y adaptador en el Backend Core.

3.7. Inicialización y enrutamiento

La inicialización del servicio se realiza mediante `LeanWorkerLauncher`. El enrutamiento no se basa en endpoints REST, sino en colas de Celery (por ejemplo, `lean_queue`) definidas en `LeanWorkerConfig`. En este modelo, el `LeanTasksController` registra manejadores de comandos explícitos, como `verify_snippet` y `verify_project_files`. Esta interfaz orientada a tareas evita ambigüedad de entrada y favorece contratos de payload fuertemente tipados por operación.

El uso de comandos discretos también permite establecer políticas diferenciadas de tiempo de ejecución, validación de parámetros y trazabilidad por tipo de tarea. Así, el worker conserva un comportamiento predecible incluso cuando conviven validaciones de distinta complejidad en la misma infraestructura.

3.8. Servicios

`LeanVerificationService` funciona como envoltorio directo del runtime formal. Sus responsabilidades incluyen localizar el ejecutable adecuado (`find_lean_executable`), lanzar procesos del sistema operativo y extraer resultados semánticos mediante `parse_theorem_info` y `parse_lean_messages`. Este último punto es especialmente relevante: la salida nativa de Lean no siempre es apta para consumo directo por API, por lo que el parseo estructurado transforma mensajes técnicos en información utilizable por el resto de la plataforma.

La estandarización de salida evita acoplar el Frontend SPA o el Backend Core a detalles internos del compilador. En consecuencia, cambios de formato en la herramienta formal se aíslan dentro del worker, preservando estabilidad de contratos externos.

3.9. Gestión de entorno temporal

Para validación de proyectos completos, `ProjectVerificationService` administra espacios de trabajo temporales (por ejemplo, `workspace_tmp_by_file_map`) y configura variables

como `LEAN_PATH`. También impone *timeouts* estrictos (referencia operativa de 90 segundos) para evitar que pruebas infinitas, recursiones patológicas o entradas mal diseñadas bloqueen el worker indefinidamente.

Esta política de aislamiento temporal y límites de ejecución cumple tres objetivos: proteger recursos del host, evitar degradación acumulativa del clúster y ofrecer tiempos de respuesta máximos acotados para la capa de orquestación. En un sistema orientado a eventos, estos límites son indispensables para mantener cola saludable y prevenir efecto cascada de retrasos.

3.10. Normalización

El `CompilerAdapterService` convierte la salida de Lean a un **JSON** uniforme y consumible por el Backend Core. Esta etapa cierra la frontera técnica entre ejecución formal y orquestación de dominio: el worker se especializa en ejecutar y traducir, mientras el backend se especializa en decidir, persistir y publicar estados.

En síntesis, **class-02** muestra un validador formal robusto y escalable, con entrada por colas, ejecución aislada, control de tiempos, parseo semántico y salida estandarizada. El valor arquitectónico de este diagrama radica en su claridad de responsabilidades: la verificación formal queda encapsulada en un microservicio desacoplado del tráfico interactivo de la plataforma.

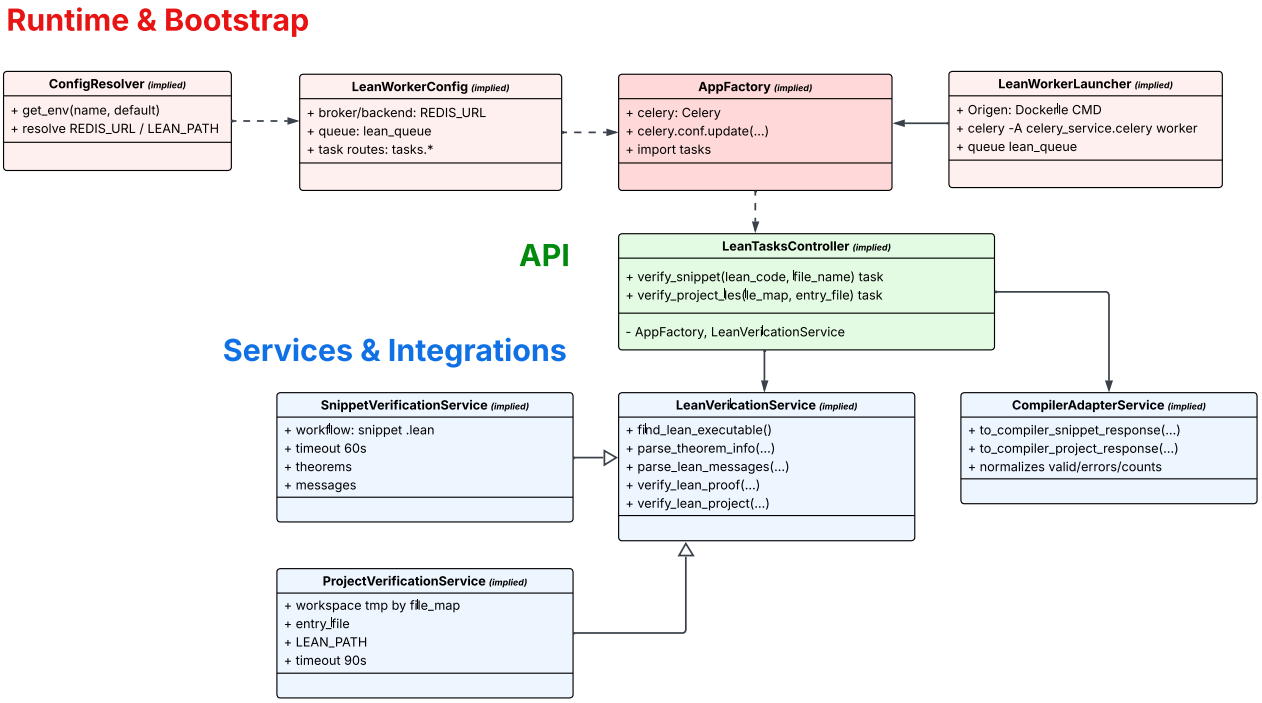

Figura 3.2: Diagrama de clases del Lean Server (microservicio)

3.11. Diagrama de clases del Frontend SPA

El diagrama **class-03** presenta una **SPA (Single-Page Application)** estructurada de forma reactiva y fuertemente tipada, donde la navegación, el estado asíncrono y la interacción con validaciones formales se integran en una sola experiencia de usuario. A nivel arquitectónico, este cliente no es un simple consumidor de endpoints, sino un coordinador de flujos de trabajo complejos sobre grafos de demostración colaborativa.

3.12. Arquitectura de interfaz y composición de rutas

La composición visual se organiza con un enrutador jerárquico (`RoutesRegistry` con `<router-outlet/>`) contenido dentro de un `ShellComponent`. El shell define el marco de navegación persistente y desacopla estructura de pantalla respecto a la lógica de cada página de dominio. Este patrón permite escalar vistas sin duplicar infraestructura de menú, control de sesión y layout general.

El valor de esta decisión es doble: mejora mantenibilidad del código frontend y ofrece una experiencia consistente para el usuario, aun cuando cambie de contexto entre módulos de proyecto, validación o administración.

3.13. Componentes de dominio y operaciones principales

Dentro del conjunto de páginas, `WorkspacePageComponent` actúa como núcleo funcional. Según el diagrama, ejecuta tareas como `loadGraph`, `parseGoalImports`, `verifyNode` y `solveNode`. Estas acciones evidencian que el frontend participa activamente en la preparación de contexto para validación formal, no únicamente en la presentación de resultados.

Por ejemplo, `loadGraph` organiza la visualización del estado lógico del proyecto; `parseGoalImports` prepara dependencias requeridas para compilar metas; y `verifyNode/solveNode` disparan operaciones de verificación y resolución con retorno asíncrono. La combinación de estas capacidades permite al usuario iterar sobre pruebas matemáticas con retroalimentación gradual y trazabilidad de cambios.

3.14. Estado asíncrono y observabilidad en tiempo real

Dado que la validación formal puede tardar desde segundos hasta intervalos mayores, el frontend implementa un patrón **Observer** a través de `TaskStreamService`. Este servicio mantiene una conexión persistente de **SSE** usando `EventSource` para recibir eventos del ciclo de vida de tareas: encolado, ejecución, éxito, error o expiración.

Con ello, la interfaz evita bloqueos visuales, elimina necesidad de sondeo agresivo y mejora transparencia del proceso para el usuario final. Desde el punto de vista arquitectónico, SSE funciona como puente entre la frontera asíncrona de colas y la experiencia interactiva de la SPA.

3.15. Capa de comunicaciones y seguridad

`HttpGatewayService` centraliza llamadas de red y construye cabeceras de autorización con JWT (por ejemplo, `authHeaders`). Este gateway se comunica con la API REST principal y con rutas de consulta de estado de workers cuando corresponde. Al concentrar acceso HTTP en un servicio único, el frontend reduce duplicación, mejora trazabilidad de errores y facilita aplicar políticas homogéneas de autenticación, reintentos y manejo de respuestas.

En resumen, **class-03** describe un frontend con responsabilidades claramente delimitadas: shell y rutas para estructura, componentes de dominio para flujo matemático, stream asíncrono para visibilidad de tareas y gateway de red para seguridad y consistencia operacional. Esta arquitectura de cliente está alineada con la naturaleza distribuida del sistema y con el requisito de interacción continua durante procesos de validación costosos.

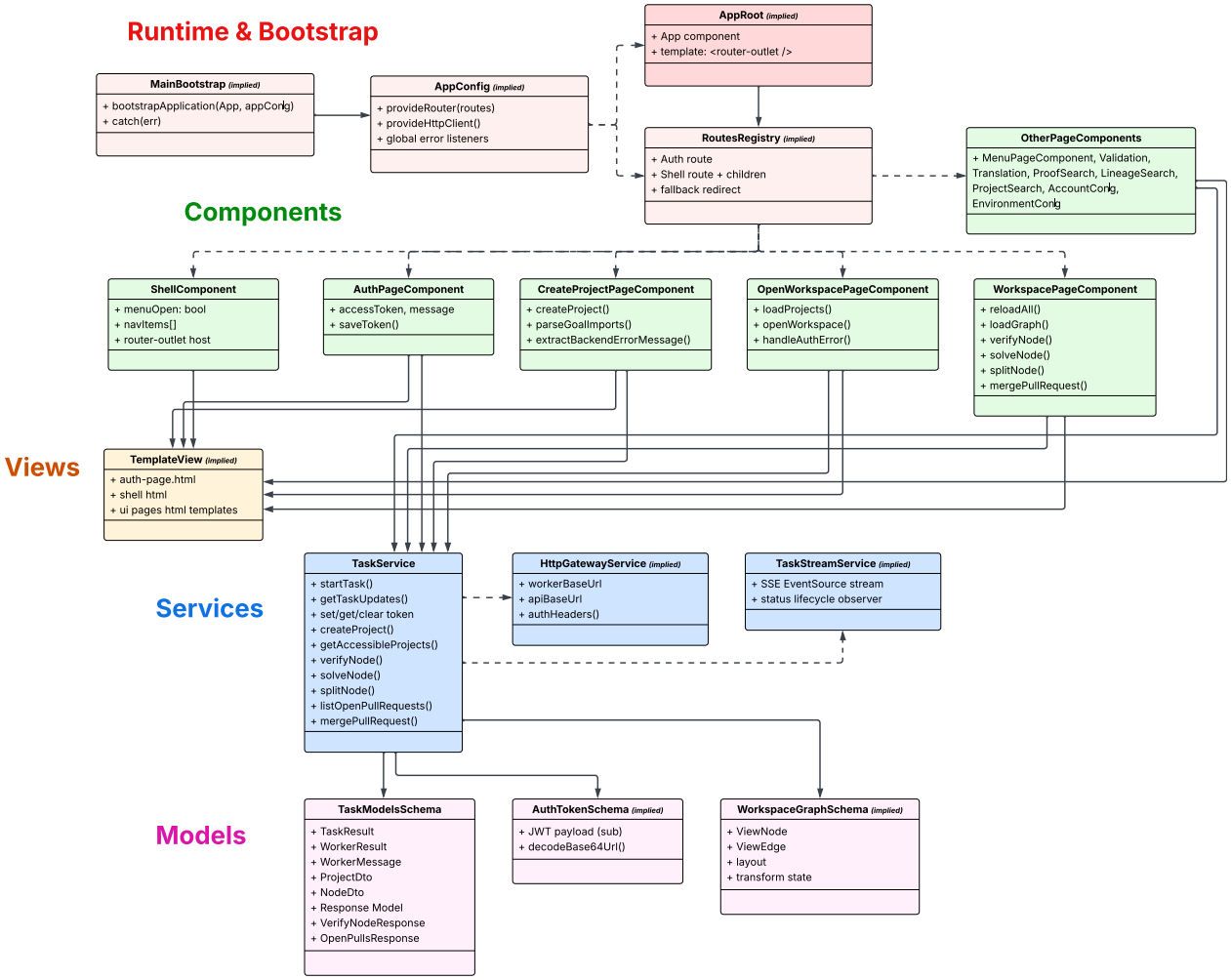

Figura 3.3: Diagrama de clases del Frontend

4. Diagrama de calles

El diagrama **street-01** muestra la topología de despliegue y las fronteras de comunicación de CoProof. A diferencia de los diagramas de clases, aquí el foco no es la estructura interna de objetos, sino el tránsito de información entre actores, contenedores y servicios. Esta perspectiva es esencial para validar propiedades no funcionales del sistema: latencia percibida, tolerancia a fallos, escalabilidad y aislamiento de cargas.

4.1. Cliente a Backend Core

La primera frontera conecta al usuario (Frontend SPA) con el Backend Core mediante **REST/HTTP** sobre **JSON**. En esta capa, el backend resuelve autenticación y autorización, incluyendo el flujo **OAuth** con GitHub, además de persistir estado estructural del dominio en PostgreSQL mediante ORM. Todo intercambio síncrono aquí está orientado a operaciones de control: abrir proyectos, consultar estructura de nodos, solicitar acciones de validación y recuperar estados de negocio.

Mantener esta frontera separada de la validación formal asíncrona permite tiempos de respuesta más estables en navegación y edición, aun si existen múltiples validaciones activas en segundo plano.

4.2. Backend Core a Lean Worker

Cuando una operación requiere validación formal, el Backend Core no ejecuta Lean directamente. En su lugar, delega el trabajo a través del `CompilerClient` (gateway interno de compilación), que emite un mensaje estructurado hacia Redis actuando como *Message Broker* y *Result Backend*. Este paso materializa el desacoplamiento arquitectónico: la petición del usuario se transforma en trabajo en cola con identidad propia y ciclo de vida independiente.

La ventaja clave es operativa: una caída temporal del worker no cancela la operación; simplemente retrasa su consumo. El sistema conserva así resiliencia frente a fallos del validador formal sin comprometer el canal principal de interacción.

4.3. Resolución y retroalimentación

El Celery Worker consume tareas de `lean_queue` mediante un patrón *pull*, ejecuta adaptadores de verificación de Lean y deposita resultados en Redis. Desde allí, el Backend Core consulta o recibe estados actualizados y los reenvía al Frontend SPA, incluyendo canales de eventos para actualización en tiempo real. Esta cadena de retorno completa el ciclo de una tarea asíncrona sin bloquear al usuario durante compilación.

El diagrama enfatiza que la experiencia de usuario no depende de una conexión de larga duración con el proceso de compilación; depende de un contrato de estado observable y consistente entre colas, backend y stream de eventos.

4.4. Escalabilidad

La topología también documenta la ruta de evolución del sistema. Integrar un traductor **NL2FL** o **agentes de IA** no exige rediseñar la arquitectura principal: basta con añadir contenedores de microservicios que escuchen nuevas colas (por ejemplo, `nl2fl_queue`, `agent_queue`) y crear un cliente adaptador adicional en el Backend Core, análogo a `CompilerClient`. Con ello, la orquestación permanece centralizada mientras la carga computacional se distribuye por dominio.

En términos de ingeniería de software, este diagrama de calles valida que CoProof está preparado para crecer por especialización de servicios, manteniendo límites claros entre coordinación, persistencia y ejecución formal. La consistencia de esta topología con los diagramas de clases confirma un diseño integral: el modelo lógico de software y el modelo físico de despliegue cuentan la misma historia arquitectónica.

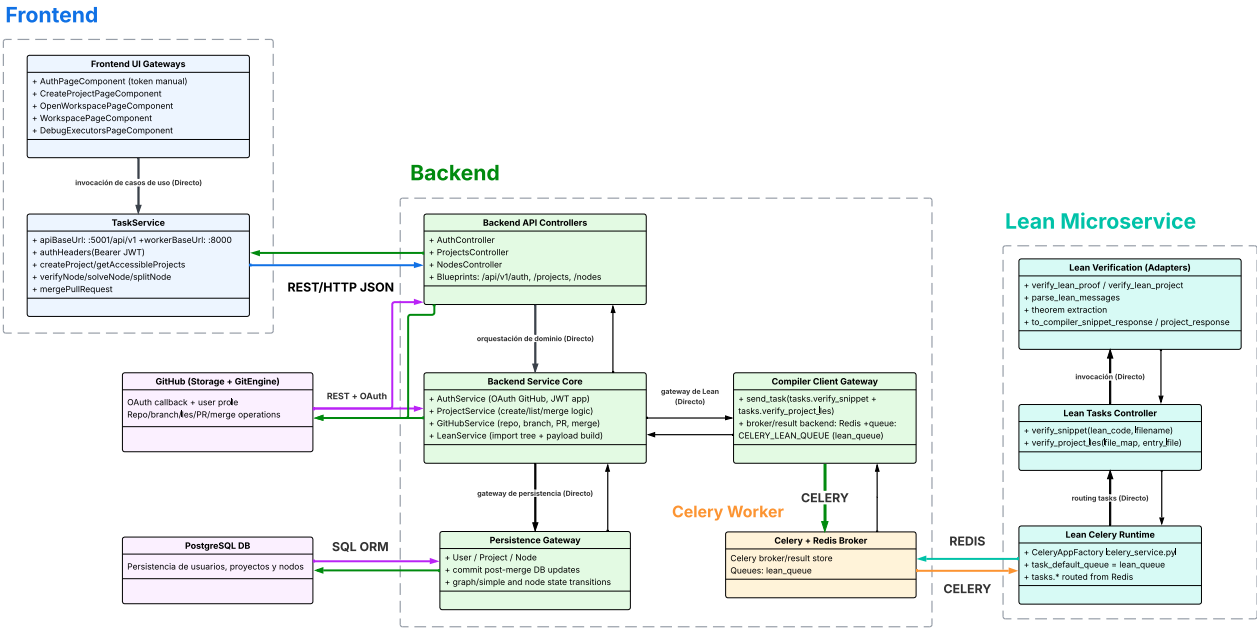

Figura 4.1: Diagrama de calles V1 (general)

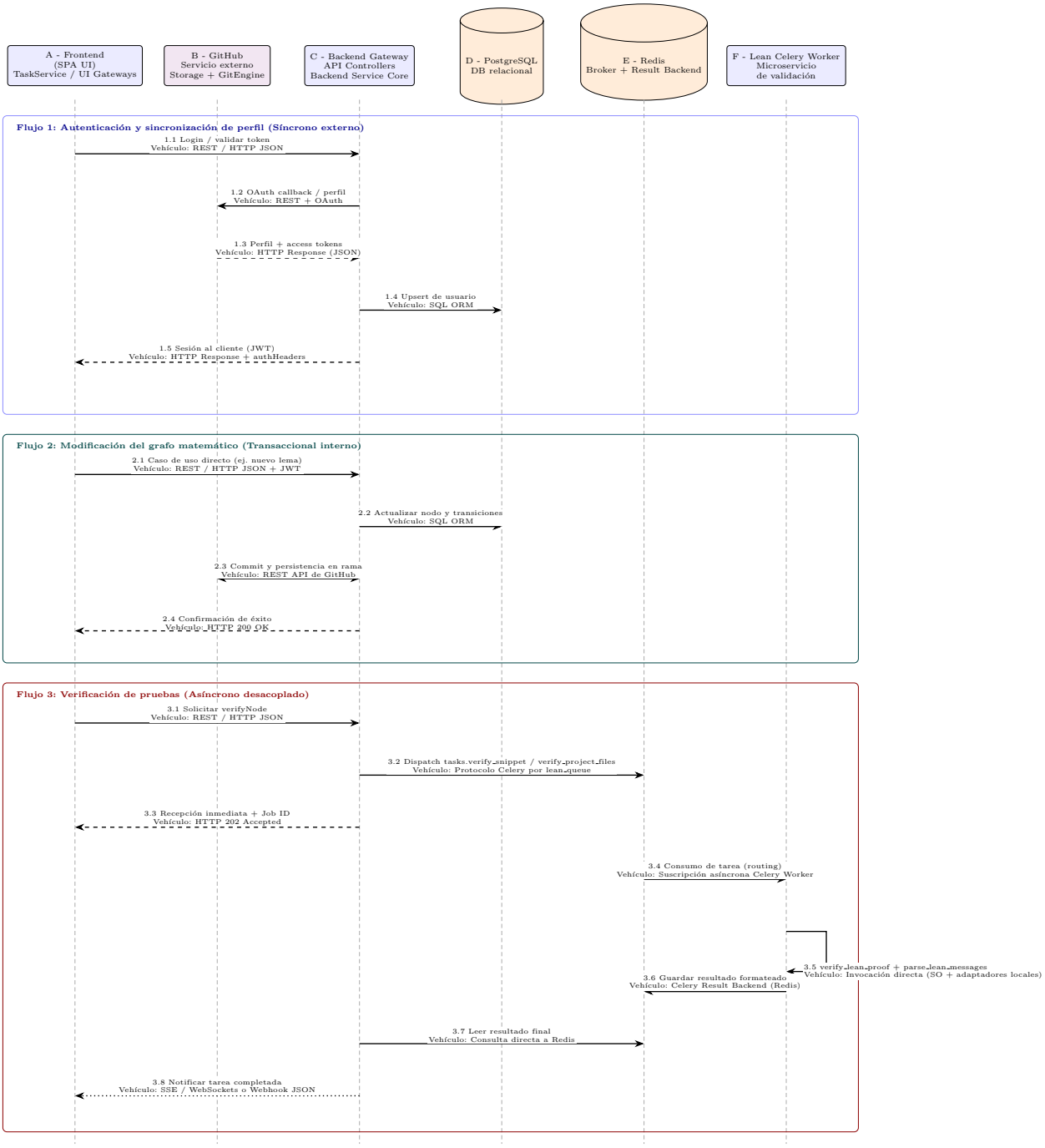

Figura 4.2: Diagrama de calles V2 (de flujos)